TDT4258 Low-Level Programming
Autumn 2026

Lab assignment 3

Embedded Device Programming

Handout: Wednesday, 23rd September 2026, 17:00
Deadline: Friday, 23rd October 2026, 17:00

Teaching Assistants: Joakim Borge Hunskår, Eskild Sæther
and Viktor Amund Holsvik
Assignment Coordinator: Andreu Girones

Pre-amble

The labs are here for you to deepen your understanding of concepts taught in the lecture. The goal is that you not only develop a theoretical understanding of the matter, but also develop the technical skills to apply it in practice.

Each lab has a **main project**, but we also provide optional exercises for those who want to go beyond the mandatory exercise. To collect points for a lab, you only need to **hand in the solution to the main project**.

The optional exercises are for possible extended learning and maybe even your entertainment.

The lab exercises are compulsory activities in the course. **You need to collect 27 points in total to be allowed to take the exam**. We assess the lab assignments such that a fully approved solution will get 10 points, and we hope that will be the normal case. Submissions with significant defects might get a reduced number of points.

As the assignments are part of the evaluation, they are subject to NTNU's plagiarism rules [1]. We have tools at our disposal and will run all submissions through plagiarism checkers. Copying code from current or past students is considered plagiarism. Hence, we advise you to not share code to prevent situations where we have to find out who copied from whom.

While copying each other is disallowed, we still encourage student discussions about your solutions. This will allow you to explore alternative approaches and solutions and learn about the advantages and challenges of particular implementations. **As a rule of thumb: Sharing and discussing ideas is fine, sharing code is not.**

1 Description

In this lab, you will program an embedded device and interact with the attached hardware. You are going to set up a Raspberry Pi 4 and use it to run a simplified version of TETRIS. The main goal of this assignment is to output the game state on a LED matrix and capture the joystick input to allow a user to play the game.

We assume that you have received a Raspberry Pi 4 with a Sense hat from the course TAs, and filled out the form associated with this equipment. You must follow the RPi 4 with Sense Hat Set Up guide[2] in full detail to be able to do the lab.

[1] https://i.ntnu.no/wiki/-/wiki/English/Cheating+on+exams

[2] Canvas →Lab3 →How to Setup RPi4 w/sense hat

2 Main Task: Play TETRIS on the Sense Hat

You find from the Lab3 page at Canvas the source code for a simple implementation of the TETRIS game, which should run out of the box on the Raspberry Pi 4 console and can be controlled with the arrow keys of your keyboard (Enter exits the game). Your task is to display the playing field in a colorful way on the Sense Hat LED matrix and make it playable with the Sense Hat joystick. You are not allowed to use any libraries apart from standard libraries and you are not allowed to reuse code that you find somewhere on the internet.

Note: You can extend the provided code to your liking, including data structures used for the game logic. However, some limitations apply:

- You are not allowed to remove or change the way the game is outputted on the console.
- You are not allowed to change the game logic, timings and speeds.
- You are allowed to change data structures within the game logic as long as they do not alter the game itself.

Task Summary

- Display the playing field on the Sense Hat 8x8 RGB LED matrix. Do not display anything else than the tiles, so no scores or other statistics. A requirement for this implementation is to memory map the frame buffer of the LED matrix (documentation below).
- The game must be playable with the Sense Hat joystick, such that a left press moves the current tile to the left, a right press to the right and a down press drops it to the bottom. Pressing up toggles pause. While paused, the playing field is completely frozen, so no pieces can move or drop. Pressing up again resumes the game. A center press must exit the game. The tile must also be continuously moved when a joystick is continuously pressed.
- Colourise the playing field such that each tile has a color that must stay the same during the entire time it is visible on the LED matrix. You have to vary the colors used so that not all tiles on the playing field have the same color. You are free in how your program chooses colors, but pay attention that all colors must be well distinguishable.

2.1 Documentation to Read

To interact with the Sense Hat you need to familiarize yourself with some fundamental concepts of using devices in Linux. You do not need to set up anything special, as your operating system image comes with all the necessary drivers and configurations. The LED Matrix of the Sense Hat is a configuration of 8x8 RGB LEDs of the type RGB565. That means that the color depth of each pixel is 16 bits. The red color channel has 5 bits, the green has 6 bits, and the blue has 5 bits, exactly in this order. The LED Matrix comes with a Linux kernel driver that exposes it as a normal framebuffer in the system with the identification 'RPi-Sense FB'.

To interact with a Linux framebuffer you need to read the following documentation carefully:

- https://www.kernel.org/doc/html/latest/fb/framebuffer.html up to and including the Programmers View of `/dev/fb`
- https://www.kernel.org/doc/html/latest/fb/api.html describes the `ioctl` interface

Note: You cannot assume that the framebuffer always has the same path or that there is only one single framebuffer on your system. Make sure you check the identification of the framebuffer before using it and exit with an error if you cannot find the `RPi-Sense FB`. Further, your LED Matrix is a fixed screen, so you are not required to check or change the variable screen information.

The joystick of the Sense Hat acts as a normal input device where each direction is mapped to the up, left, right, and down key. Pressing the joystick emits the enter key. You will find a raw interface for the joystick device on your system. The identification of the input device is 'Raspberry Pi Sense HAT Joystick'.

To interact with the Linux event system which exposes the joystick inputs, you need to read the following documentation:

- https://www.kernel.org/doc/html/latest/input/input.html - pay special attention to the **Event Interface**.
- https://www.kernel.org/doc/html/latest/input/event-codes.html - focus on the EV_KEY event.

Note: You cannot assume that the joystick has always the same path or that there is only one on your system. Check the identification before

using it and exit with an error if you do not find `Raspberry Pi Sense HAT Joystick`. When an input event is ready, it might not be the only event in the queue. There can be multiple input events ready for reading. You have to check and read input events in a non-blocking way, so that the game does not pause between input events. Furthermore, the tiles must continuously move when joystick is continuously pressed.

Familiarize yourself with the following functions that are helpful or necessary to interact with devices in Linux:

- `mmap` (https://man7.org/linux/man-pages/man2/mmap.2.html) and `munmap` (https://man7.org/linux/man-pages/man3/munmap.3p.html) to map and unmap device memory into the virtual address space.
- `ioctl` to perform special device actions, i.e. setting and retrieving device settings. https://man7.org/linux/man-pages/man2/ioctl.2.html https://www.kernel.org/doc/html/latest/driver-api/ioctl.html
- `poll` to poke if there is anything to do https://man7.org/linux/man-pages/man2/poll.2.html

3 Optional Tasks

- **A [EASY]:** Light up the entire LED Matrix of the Sense Hat with a single color. Change the color with the press of the joystick device (remember that this emits the enter key).
- **B [MEDIUM]:** (Generative AI) **mini-Pac-Man**
  This task is inspired by the classical computer game **Pac-Man**. Its development began early in 1979 and took one year and five months to complete, the longest for any video game to that date. See if you with help of agentic AI can develop a *prototype* of mini-Pac-Man.
  The task: Initially all 64 LED's should be blue, and a red blinking LED should be at position (4,4). Every 5th second a green LED (energy pill) pops up at some random position, and it will stay there for 5 seconds. Start with 5 green LEDs. The player should move the red LED with keyboard arrow-keys towards a green LED before it gets blue again, and if reached you get 1 point. Every half minute the life of the green LEDs should be decreased by one second. See how many pills you can eat within two minutes, and write the high-score to a file. If you need more specifications, add them.

Use plan-mode in Cline and ask Mistral Medium for a step by step plan for developing the game. Go to Cline Act mode at start implementing the various steps. Interact with cline, review code, discuss, and correct but also give it a chance to work on its own initiative. See TDT4258 - GenAI example 4 for suggestions for how to proceed. Get a friend to test the game and ask for feedback. If you think it works well make a short video, give your professor access to it and the video will *perhaps* be shown in the break of last lecture :-).

- **C [HARD]:** The simple TETRIS implementation uses a single simple shape. Can you extend it to multiple different shapes?

4 Submission and assessment

Submit your **commented C code file** `stetris.c` before the deadline on Canvas. Ensure you haven't changed the console output of the game and that it still runs the provided game logic in the same speed. Make sure that your program compiles and executes before you submit.

We expect all submissions to meet the following requirements:

1. The submission is on time. We provide enough time to solve the labs, but we expect you to start early.
2. The program follows the outlined requirements.
3. You have submitted an *"AI-statement"* for this lab exercise. It will be anonymized and will not influence on the assessment — but will help us all to gather experience with the use of generative AI in the course. This is further explained in lecture 0 and on Canvas.

Failing to meet any of these requirements will result in the number of points collected by the submission to be reduced, depending on the seriousness of the defect(s). No submission gives zero points.

Commenting on your code and keeping it tidy is very important. Helping us understand what you did, supports us in assessing your work – we can only give points for what we understand.

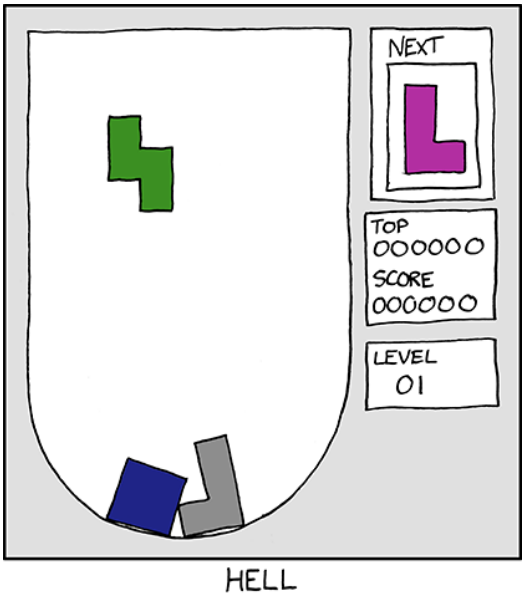
5 Similarity Checking and Plagiarism

You must submit your **own work**. You must write your **own code** and **not copy** it from anywhere else, including your classmates, internet, and automated tools.

6 Questions

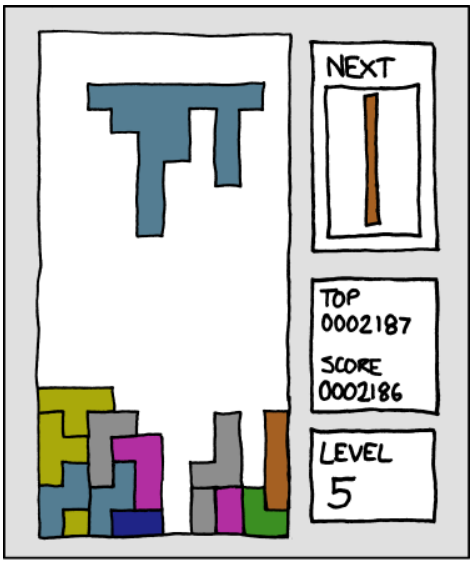
If you have any questions about this assignment, we encourage you to ask the question on the discussion forum provided by the course. By that, you also help other students who have the same questions in the future.

Source: https://xkcd.com/724/



HELL

Source: https://xkcd.com/888/



HEAVEN